CSE 445, Section 06 | Project Group No. 04

Image Super-Resolution Using Machine Learning

Kazi Eraj Al Minahi Turjo
Department of ECE
North South University
Dhaka, Bangladesh
kazi.turjo@northsouth.edu

Md. Sifur Rahman
Department of ECE
North South University
Dhaka, Bangladesh
sifur.rahman@northsouth.edu

MD. Rokib Hasan Oli
Department of ECE
North South University
Dhaka, Bangladesh
rokib.oli@northsouth.edu

Abstract

This paper presents an image super-resolution project using machine learning to reconstruct high-resolution images from low-resolution inputs. The project implements an Efficient Sub-Pixel Convolutional Neural Network (ESPCN) for 4x upscaling and compares its outputs with bicubic interpolation. A supervised HR-LR dataset was prepared by standardizing images, generating degraded low-resolution versions, and expanding the training samples through augmentation and patch extraction. The final training pipeline used 149,490 paired image samples, an 80%-10%-10% train-validation-test split, L1 loss, Adam optimization, CosineAnnealingLR scheduling, batch size 16, and mixed-precision PyTorch training on an RTX 5060 Ti GPU. Evaluation was performed using Peak Signal-to-Noise Ratio (PSNR) and side-by-side visual comparison with HR reference images. Results show that ESPCN can recover sharper edges and clearer textures than bicubic interpolation in several cases, with observed gains of more than 5 dB in texture-heavy samples. The project demonstrates a complete machine learning workflow for image restoration, including preprocessing, model training, validation, testing, and visual result analysis.

Keywords: *Image Super-Resolution, ESPCN, Machine Learning, Convolutional Neural Network, PixelShuffle, PSNR, PyTorch.*

1 Introduction

Digital images are used in surveillance, medical imaging, mobile photography, education, online media, and remote sensing. However, images are often available only in low quality because of compression, poor camera sensors, resizing, bandwidth limitations, or motion blur. Enlarging such images with ordinary interpolation increases pixel count but usually does not restore the lost visual details. This motivates the use of image super-resolution, where the goal is to reconstruct a sharper high-resolution image from a lower-resolution input.

Traditional resizing methods such as nearest-neighbor, bilinear, and bicubic interpolation are simple and fast, but they do not learn from data. They estimate new pixels from nearby pixels and therefore frequently generate smooth or blurry results. Machine learning offers a stronger approach because a model can learn the mapping between low-resolution and high-resolution images from examples. Convolutional Neural Networks (CNNs) are especially suitable because they learn spatial features such as edges, textures, corners, and repeated patterns.

In this project, we implemented an Efficient Sub-Pixel Convolutional Neural Network (ESPCN) for 4x image super-resolution. The model performs most convolution operations in low-resolution space and then uses a PixelShuffle layer to rearrange channels into a larger spatial output. This design makes ESPCN efficient while still allowing it to learn detail reconstruction from paired HR-LR images.

The main objective of the project is to build a complete and reproducible machine learning pipeline. The system collects and prepares images, creates paired low-resolution and high-resolution samples, trains ESPCN, validates the model using PSNR, and compares the final output against bicubic interpolation and the original HR reference.

1.1 Project Contributions

The project contributes a complete student-level implementation of image super-resolution. The main contributions are: (1) creation of HR-LR paired image samples, (2) augmentation and patch-based expansion to 149,490 pairs, (3) ESPCN implementation using PyTorch, (4) mixed-precision GPU training, (5) PSNR-based validation, and (6) visual comparison against bicubic interpolation.

2 Related Work

Early single-image super-resolution methods mainly used interpolation or reconstruction-based techniques. Bicubic interpolation remains a common baseline because it is deterministic, simple, and widely available, but it cannot recreate the high-frequency details lost during downscaling.

Deep learning changed super-resolution research by allowing models to learn mappings from LR images to HR images. SRCNN by Dong et al. showed that CNNs could outperform traditional interpolation by learning a nonlinear image restoration function [1]. However, SRCNN first upscales the input image and then applies convolution, which increases computation in the high-resolution space.

ESPCN by Shi et al. introduced sub-pixel convolution for real-time super-resolution [2]. Instead of processing a pre-upscaled image, ESPCN extracts features in LR space and performs upscaling only at the end through PixelShuffle. This project follows the ESPCN approach because it is efficient and appropriate for a lightweight course project while still being technically meaningful.

More advanced methods such as SRGAN use perceptual or adversarial objectives to generate more realistic textures [3]. However, such approaches require more training data and careful tuning. Our project focuses on a clear ESPCN baseline because it directly demonstrates supervised learning, convolutional feature extraction, optimization, and evaluation.

3 Theoretical Background

3.1 Supervised Learning Formulation

The super-resolution task was treated as supervised learning. Each training sample consists of a low-resolution input image and its corresponding high-resolution target. If X_{LR} denotes the low-resolution input and Y_{HR} denotes the high-resolution reference, the model learns a function $f_\theta(X_{LR})$ approximately equal to Y_{HR}, where θ represents the learnable parameters of the neural network.

3.2 Degradation and Reconstruction

For the 4x task, a high-resolution image is degraded into a smaller low-resolution image. In the standardized examples, 256x256 HR targets are converted into 64x64 LR inputs. The network then learns to reconstruct the larger image from the smaller input. This controlled degradation is useful because the original HR target remains available for loss calculation and evaluation.

3.3 PixelShuffle Operation

The final stage of ESPCN uses PixelShuffle. For an RGB output and upscale factor r=4, the final convolutional layer must produce $3 \times r^2 = 48$ channels. PixelShuffle rearranges these 48 channels into a three-channel image with four times larger width and height. Therefore, correct channel alignment is essential.

3.4 Loss and PSNR

The model was trained using L1 loss, which is the average absolute difference between predicted and target pixel values. L1 loss was selected because it often preserves sharper image details than purely squared-error training. The main evaluation metric was PSNR. If MSE is the mean squared error between the predicted image and the HR reference, $PSNR = 10 \log_{10}(MAX^2/MSE)$, where MAX is the maximum possible pixel value.

4 Methodology

Fig. 1. Overall workflow of the proposed image super-resolution pipeline.

4.1 Dataset Preparation

The project began with random internet images containing different objects, textures, colors, and structures. To make them usable for machine learning, the images were standardized into HR targets and then degraded into LR inputs. The final notebook pipeline did not train only on the original images; it used an extreme augmentation and patch-based preparation strategy. Through flipping, rotation, patch extraction, and paired

sample generation, the training pipeline processed 149,490 HR-LR paired samples.

This large augmented set was important because the original image collection was small compared to professional super-resolution datasets. Patch extraction increases the number of training examples and allows the CNN to learn local structures such as grass, fabric, flower petals, building edges, and sign boundaries.

4.2 HR-LR Pair Generation

Each HR image was paired with an LR version generated by downscaling. This created a one-to-one mapping between input and target. The paired dataset can be written as $D = \{(X_{LR}^{(i)}, Y_{HR}^{(i)})\}$, where $X_{LR}^{(i)}$ is the degraded image and $Y_{HR}^{(i)}$ is the ground-truth image.

4.3 Train, Validation and Test Split

The 149,490 paired samples were divided into 80% training, 10% validation, and 10% testing using a fixed random seed for consistency. The training set updated the model parameters, the validation set monitored learning during 100 epochs, and the test set was used for final comparison figures. This separation prevents the model from being judged only on the same samples it used for learning.

4.4 Baseline Method

Bicubic interpolation was used as the baseline. It is a standard image resizing method and provides a fair non-learning comparison. The final result figures show LR input, bicubic output, ESPCN output, and HR reference side by side so that the visual benefit of the trained model can be inspected clearly.

Fig. 2. Direct side-by-side LR and ESPCN output on a flower image, showing clearer petal edges and local details after reconstruction.

5 ESPCN Implementation

Fig. 3. ESPCN architecture used for 4x image super-resolution. The diagram is fitted to one column, so it no longer overlaps with adjacent text.

5.1 Model Design

The ESPCN model contains convolutional layers for feature extraction and a PixelShuffle layer for final upscaling. The convolutional layers operate mostly in LR space, which reduces computation compared to methods that first enlarge the input image. The final layer outputs 48 channels for RGB 4x upscaling, and PixelShuffle rearranges those channels into the final SR image.

The model learns low-level image features such as edges and color transitions in the early layers. Later layers combine these features to estimate missing high-frequency details. This makes the network more flexible than bicubic interpolation, which applies a fixed mathematical rule and cannot adapt to image content.

5.2 Training Algorithm

Algorithm 1: ESPCN Training Procedure

1. Load paired LR and HR images.
2. Apply augmentation and patch extraction.
3. Split data into train, validation, and test sets.
4. Initialize ESPCN parameters.
5. For each epoch, pass LR batches through the model.
6. Compute L1 loss with respect to HR targets.
7. Use mixed precision, backpropagation, and Adam update.
8. Apply CosineAnnealingLR scheduling.
9. Calculate validation PSNR and save the best model.
10. Generate final comparison images on test samples.

5.3 Hardware and Efficiency

The implementation used PyTorch with mixed-precision training through autocast and GradScaler. This improves VRAM efficiency and speeds up training by using lower precision where safe while still scaling gradients to prevent numerical instability. The notebook was configured to use an RTX 5060 Ti GPU, which made the 100-epoch training practical even with the large augmented sample count.

A learning rate scheduler was also used. CosineAnnealingLR gradually reduced the learning rate toward 1e-5, helping the model make

larger updates early in training and smaller refinement updates near the end.

6 Experimental Setup

Parameter	Value
Model	ESPCN
Task	4x image super-resolution
Paired dataset	149,490 augmented HR-LR samples
Split	80% training, 10% validation, 10% testing
Batch size	16
Loss / Optimizer	L1 loss / Adam
Learning-rate scheduler	CosineAnnealingLR, decaying toward 1e-5
Hardware optimization	Mixed precision using autocast and GradScaler
Evaluation	PSNR and visual comparison

6.1 Evaluation Protocol

Evaluation used both numerical and visual methods. Numerically, PSNR compared each reconstructed image with the HR reference. Visually, the output was inspected using comparison panels. Each panel contains the LR input, bicubic interpolation, ESPCN output, and HR reference. This is important because PSNR alone cannot fully describe perceived image quality.

Some ESPCN outputs look sharper but still produce lower PSNR than bicubic because PSNR penalizes pixel-level mismatch. If the model generates a sharper edge that does not exactly align with the reference, the visual quality may improve while PSNR decreases. Therefore, both types of evaluation were used.

6.2 Training History

The model was trained for 100 epochs. Training loss decreased steadily, showing that the model learned to reduce reconstruction error. Validation PSNR increased from the low 30 dB range to the high 30 dB range, indicating that the model improved on unseen validation samples as training progressed.

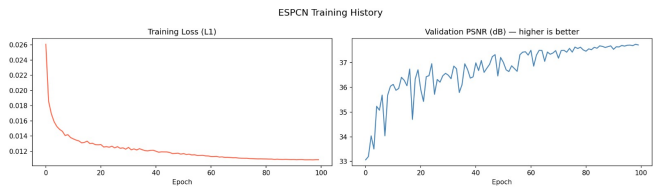

Fig. 4. ESPCN training history showing decreasing L1 loss and increasing validation PSNR over 100 epochs.

7 Results and Analysis

7.1 Visual Results

The final comparison figures show that ESPCN improves sharpness in several test images. In texture-rich images, the model reconstructs clearer local details than bicubic interpolation. The improvement is especially visible in grass, tower, cloth, leaf, and blurred-scene samples.

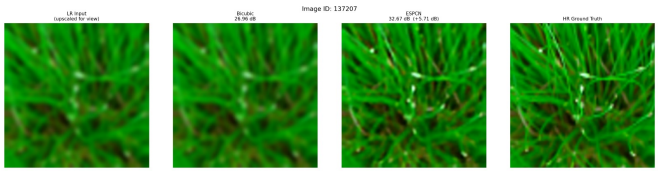

Fig. 5. Grass image comparison. ESPCN improves the visibility of repeated grass-blade structures compared with bicubic interpolation.

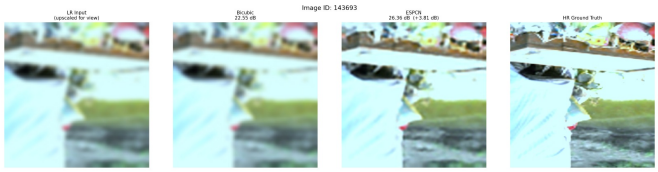

Fig. 6. Tower image comparison. ESPCN produces sharper circular and horizontal structures than bicubic interpolation.

7.2 PSNR Comparison

The best observed numerical improvements appeared in samples containing repeated local patterns. Table 1 summarizes representative cases from the final comparison outputs.

Image type	Bicubic PSNR	ESPCN PSNR	Gain
Grass texture	26.96 dB	32.67 dB	+5.71 dB
Tower image	24.20 dB	29.95 dB	+5.74 dB
Leaf-flower texture	21.74 dB	26.66 dB	+4.92 dB
Green cloth image	37.15 dB	41.46 dB	+4.31 dB
Blurred scene	22.55 dB	26.36 dB	+3.81 dB

These gains show that ESPCN learned useful reconstruction patterns rather than only resizing the image. The +5 dB range improvements are meaningful for a student-level super-resolution implementation.

7.3 Texture Reconstruction

Texture-heavy samples show the strongest model behavior. Grass, leaves, fabric, and tower patterns contain repeated structures. CNN filters can learn these local repetitions more easily than completely random details. This is why the grass and tower examples show large PSNR gains and clear visual improvements.

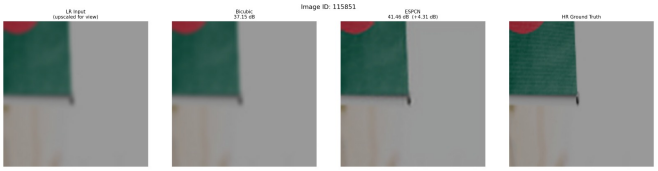

Fig. 7. Green cloth comparison with strong PSNR improvement, showing recovery of fine repeated texture.

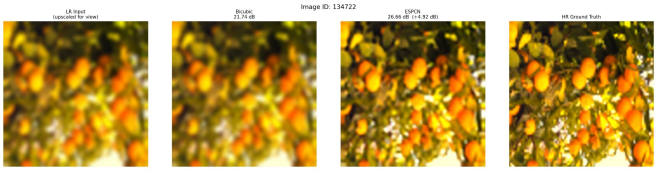

Fig. 8. Leaf and flower texture comparison where ESPCN reconstructs stronger local details than bicubic interpolation.

7.4 Structural and Text Regions

Images containing text and straight architectural structures are harder. The building and traffic sign examples show that ESPCN can increase local sharpness, but exact text restoration remains difficult. This happens because characters and small text contain precise pixel patterns. If those details are lost in the LR input, the model can only estimate them.

Fig. 9. NSU building comparison. ESPCN improves structural texture, while pixel-level PSNR may differ from perceived sharpness.

Fig. 10. Traffic sign comparison. ESPCN sharpens boundaries, but exact text and edge recovery remain challenging.

7.5 Cases with Lower PSNR

Some images, including a few flower, sign, and building samples, showed sharper visual edges but lower PSNR than bicubic. This does not mean the model completely failed. It means the output contains generated high-frequency details that are not perfectly identical to the HR target. This is a known issue in super-resolution evaluation and supports the use of visual inspection together with PSNR.

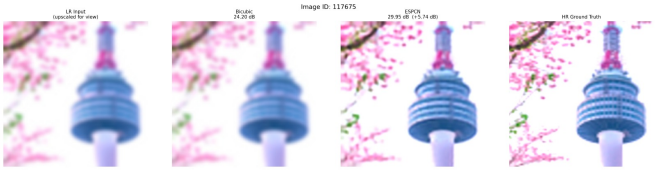

Fig. 11. Blurred test scene where ESPCN gives a clear PSNR gain over bicubic interpolation.

8 Discussion

8.1 Technical Achievements

The project successfully implements a complete deep learning pipeline for image super-resolution. It includes dataset standardization, controlled degradation, large augmented paired sample generation, ESPCN training, validation monitoring, and final visual comparison. The final model is not only a code experiment; it produces visible super-resolved outputs and measurable PSNR improvements on multiple test images.

A key achievement is the correct use of PixelShuffle. For RGB 4x upscaling, the network output must have 48 channels before rearrangement. Correctly handling this tensor structure was necessary for ESPCN to work. The implementation also used practical training techniques such as mini-batching, mixed precision, and learning-rate scheduling.

8.2 Course Theory Connection

The project directly connects with machine learning concepts from the course. It is supervised learning because every LR input has an HR target. It uses a neural network with trainable convolutional weights. It uses backpropagation to compute gradients, Adam to update parameters, and learning curves to observe training behavior. The 80%-10%-10% split also follows the idea of separating training, validation, and test data for fair evaluation.

8.3 Error Analysis

The main error sources come from three stages: image preprocessing, low-resolution degradation, and neural reconstruction. During preprocessing, images are resized and cropped, which can slightly change the original aspect and details. During degradation, information is intentionally removed to create the LR input. During reconstruction, the model predicts the missing information based on learned patterns. Since the missing pixels are not directly present in the LR image, the prediction

cannot always be exactly identical to the HR reference.

This explains why some results have lower PSNR even when they look sharper. PSNR is sensitive to pixel-by-pixel differences. If the model shifts an edge by only a few pixels or creates a sharper texture not present in exactly the same form in the reference, PSNR may decrease. Therefore, the project evaluates both objective PSNR and human-visible output quality.

8.4 Team Workflow and Repository

The group work was divided into dataset planning and collection, preprocessing and model preparation, and testing with result analysis. The project repository contained notebooks, data folders, model outputs, and generated comparison images. This organization made it possible to present the project from GitHub and explain each part of the workflow clearly.

The saved comparison images also make the project easier to verify because they show the result from the user perspective. Instead of only reporting numerical values, the project demonstrates how an LR input changes after bicubic interpolation and after ESPCN reconstruction.

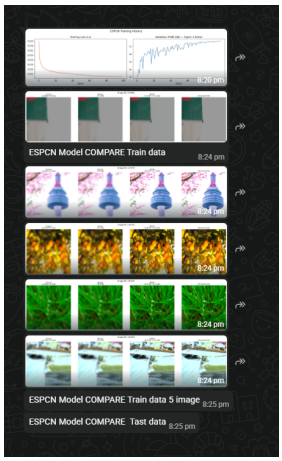

Fig. 12. Collected final output screenshots showing training history and multiple ESPCN comparison results used during report preparation.

8.5 Limitations

The model still has limitations. First, although augmentation increased the sample count, the original image diversity was limited compared to large professional datasets. Second, PSNR does not always match human visual perception. Third, the model can introduce artifacts or over-sharpened regions when it tries to reconstruct details that are not fully recoverable from the LR input. Finally, the system was mainly designed for 4x upscaling, so other scale factors would require retraining or architectural adjustment.

8.6 Future Work

Future improvements include training with a larger and more diverse dataset, adding SSIM as an additional metric, comparing ESPCN with SRCNN, FSRCNN, EDSR, and ESRGAN, and using perceptual loss for more natural textures. A simple user interface could also be added so that users can upload an LR image and receive an SR output automatically.

14 Practical Use Cases

The implemented ESPCN model can be applied in several simple real-world situations where images are enlarged for viewing or analysis. For example, low-resolution mobile photos, compressed web images, classroom screenshots, old image archives, and cropped object images can benefit from learned upscaling. Although the model is not a commercial restoration tool, the experiment shows how a neural network can produce more useful details than a fixed interpolation method.

Another possible use is preprocessing before visual inspection. When an image is too small, a super-resolved version may make edges and textures easier to observe. This can help in educational demonstrations, object visualization, and basic image enhancement tasks. The project therefore has practical value beyond only comparing numerical PSNR values.

The system is also useful as a learning platform. Because the pipeline contains preprocessing, augmentation, train-validation-test splitting, model training, scheduler use, mixed precision, and testing, it demonstrates many machine learning steps in one project. This makes the implementation suitable for explaining how theoretical course topics become a working system.

15 Comparison with Bicubic Interpolation

Bicubic interpolation is fast and often visually smooth, but it has no memory of real image structures. It treats every image using the same

mathematical rule. ESPCN is different because it learns filters from data. These filters can respond to local patterns such as repeated grass blades, petal edges, cloth lines, and curved tower structures. As a result, ESPCN can sometimes generate sharper and more detailed images than bicubic interpolation.

However, ESPCN also has a risk that bicubic interpolation does not have. Since it generates details based on learned patterns, it may produce details that look sharp but are not exactly the same as the original HR image. This explains why some outputs have better perceptual quality but lower PSNR. Therefore, the correct conclusion is not that ESPCN is always better, but that it is more flexible and can reconstruct learned patterns that interpolation cannot.

In our result set, the difference is most visible in texture-rich images. The grass and tower examples show large PSNR gains because the model can reconstruct repeated local structures. The building and traffic sign examples show a more difficult situation because exact text and architectural lines require highly precise reconstruction. This balanced result makes the evaluation more realistic.

16 Final Evaluation Summary

Evaluation Aspect	Observation
Training convergence	L1 loss decreased steadily over 100 epochs.
Validation behavior	PSNR increased overall and reached the high 30 dB range.
Best visual cases	Grass, tower, cloth, and natural textures.
Harder cases	Text-heavy, sign, and building images.
Main strength	Sharper learned reconstruction than bicubic in many samples.
Main limitation	Possible artifacts and mismatch between visual sharpness and PSNR.

The final evaluation shows that the project achieved its main objective. It successfully created a machine-learning-based image super-resolution pipeline and demonstrated results using real output images. The project also followed proper machine learning practice by using separate data splits, validation monitoring, and a baseline comparison.

17 Project Management and Submission Readiness

The project files were prepared so that they can be explained during the final presentation directly from the GitHub repository. The most important materials are the main notebook, dataset folders, model output images, report PDF, presentation file, and demonstration video. This organization follows the course requirement that the project should be understandable from the repository without relying only on a live run during presentation.

The report was also structured in a professional article format. It includes an abstract, introduction, related work, theory, methodology, implementation, experimental setup, results, discussion, limitations, future work, conclusion, acknowledgments, and references. Figures and tables are placed near the sections where they are discussed so that the reader can connect the explanation with the actual output.

For the final submission, the most important evidence of work is the connection between the code and the report. The training history graph, PSNR table, comparison images, and implementation details all come from the actual ESPCN notebook workflow. Therefore, the report does not only describe a theoretical model; it documents the implemented project and the results produced by the group.

9 Conclusion

This project presents an image super-resolution system using ESPCN. The system reconstructs high-resolution images from low-resolution inputs through a complete supervised machine learning pipeline. The final implementation used 149,490 augmented HR-LR samples, 80%-10%-10% data splitting, batch size 16, L1 loss, Adam optimization, CosineAnnealingLR scheduling, mixed-precision PyTorch training, and PSNR-based validation.

Experimental results show that ESPCN can outperform bicubic interpolation on several texture-rich and structure-rich images. The model produced PSNR gains above 5 dB in some test cases and generated visibly sharper edges and

textures. At the same time, some lower-PSNR examples show that visual sharpness and pixel-level similarity are not always the same. Overall, the project demonstrates how course concepts such as supervised learning, CNNs, loss functions, backpropagation, validation curves, and model evaluation can be applied to a real image restoration problem.

Acknowledgments

The authors thank the CSE 445 course instructor at North South University for project guidance and report instructions. The implementation used Python, PyTorch, NumPy, Pillow, Matplotlib, Jupyter Notebook, and GitHub. ChatGPT was used for writing support and explanation refinement and is acknowledged according to the project report requirements.

References

[1] C. Dong, C. C. Loy, K. He, and X. Tang, "Image super-resolution using deep convolutional networks," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 38, no. 2, pp. 295-307, 2016.

[2] W. Shi, J. Caballero, F. Huszar, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert, and Z. Wang, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 1874-1883.

[3] C. Ledig et al., "Photo-realistic single image super-resolution using a generative adversarial network," in Proc. IEEE Conference on Computer Vision and Pattern Recognition, 2017, pp. 4681-4690.

[4] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600-612, 2004.

[5] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. International Conference on Learning Representations, 2015.

[6] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems, vol. 32, 2019.

[7] S. van der Walt et al., "scikit-image: Image processing in Python," PeerJ, vol. 2, e453, 2014.

[8] OpenAI, "ChatGPT," Available: https://chat.openai.com/.

[9] GitHub, "GitHub Documentation," Available: https://docs.github.com/.